GSA

Gemeinsame Subsequenz

u with $|u| = m$ & v with $|v| = n$

gemeinsame Subsequenz von u und v ist eine Folge

$(i_1, j_1), \dots, (i_r, j_r)$:

1) $1 \le i_1 < \dots < i_r \le m$
2) $1 \le j_1 < \dots < j_r \le n$
3) $u[i_1] = v[j_1], \dots, u[i_r] = v[j_r]$

(Subsequenz:
$(i_1, j_1), \dots (i_r, j_r)$ $\quad 1 \le i_1 < \dots < i_r \le m$, $1 \le j_1 < \dots < j_r \le n$ $\Rightarrow u[i_k] \to v[j_k]$)

Calculate LCS — LCS-Graph

1) Nodes / Edges:
   Nodes: (i,j), ~~$u[1..i]$, $v[1..j]$~~ $\quad 0 \le i \le m$, $0 \le j \le n$
   Edges: I, D, R nur bei $u[i] = v[j]$

2) Bewertung Kanten:
   I / D : 0, Matches: +1 $\Rightarrow$ find max Matches!

3) Aus welchen Pfaden im LCS-Graph ergibt sich die Länge der LCS? — maximierenden Pfad!

LCS_δ-Matrix $(m+1) \times (n+1)$

$LCS_\delta(i,j)$: die Länge der längsten gemeinsamen Subsequenz für die $u[1..i]$ & $v[1..j]$ — Speicher $O(m,n)$

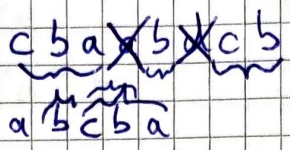
$$LCS_\delta(i,j) = \begin{cases} 0, & i = 0 \text{ or } j = 0 \\ LCS_\delta(i-1, j-1) + 1, & u[i] = v[j] \\ \max \begin{cases} LCS_\delta(i-1, j) \\ LCS_\delta(i, j-1) \end{cases}, & \text{sonst} \end{cases}$$

GSA

Maximal Matches for seq comparison

→ Common Substrings

→ One counts the minimum number of occurrences of characters in one sequence s.t. if these chars are removed, the remaining substrings are all substrings of the other sequence



Partition

Partition Ψ von v mit Respekt zu u ist eine Sequenz $(w_1, c_1, \ldots, w_r, c_r, w_{r+1})$ of substrings $w_1, \ldots, w_{r+1}$ of u and characters $c_1, \ldots, c_r$ s.t. $v = w_1 c_1 \ldots w_r c_r w_{r+1}$

$w_1, \ldots, w_{r+1}$ are submatches in Ψ.

$c_1, \ldots, c_r$ are marked characters in Ψ.

$|\Psi| = r$

mmdist(v,u) is the size of any minimal partition of v with respect to u = maximal matches distance of v and u

- k, $1 \le k \le r$, $w_k c_k$ is not a substring of u.
  left-to-right partition $\Psi_{lr}(v,u)$
- k, $1 \le k \le r$, $c_k w_{k+1}$ is not a substring of u
  right-to-left-partition $\Psi_{rl}(v,u)$

$\Rightarrow |\Psi_{lr}(v,u)| = \text{mmdist}(v,u) = |\Psi_{rl}(v,u)|$

GSA

mmdist calculation

ψ_{lr}

simple algorithm: scan $l \to r$ till prefix wc of v is found that w is a substring of u, but wc not ...

using suffix tree of u: Runij Time $O(|A| \cdot (m+n))$
Space $O(m)$

ψ_{lr}: $ST(u)$
ψ_{rl}: $ST(u^{-1})$

mmdist Metrik

mmdist is not a metric on A^*

$$mmdist(v,u) \neq mmdist(u,v)$$

Obtain a metric with:

$$mmm(u,v) = \log_2((mmdist(u,v)+1)(mmdist(v,u)+1))$$

mmm is a metric on A^*.

Runij Time $O(|A| \cdot (m+n))$ and Space $O(\max\{m,n\})$

mmdist and unit edist

Relation Alignment and Partition: $\psi \to A$ $\quad$ (w) matches R $\quad$ (c) char I

δ unit cost, A alignment of w and u. Then there is an r:

$0 \leq r \leq \delta(A)$, and a partition $(w_1, c_1, \dots, w_r, c_r, w_{r+1})$ of $v \to u$ s.t. w_1 is a prefix and w_{r+1} is a suffix of u

$\Rightarrow$ $mmdist(v,u)$ is a lower bound for the unit $edist_\delta(u,v)$

$$mmdist(v,u) \leq edist_\delta(v,u)$$

GSA

mmdist & edist

Use mmdist as a filter in contexts, where the exact edit distance is of interest only below some threshold k.

eg:

Set S of sequences. Find $\forall (s, s') \in S \times S,\ s = s'$ s.t. $edist_S(s, s') \leq k$.

First capute $mmdist(s, s')$ in $O(|s| + |s'|)$ time.

if $mmdist(s, s') > k$ ~~capute~~ do ~~look further~~ nd look at that pair!

GSA

q-gram sequence comparison

$q > 0$, q-gram of a sequence s is a substring of s of length q.

- In this model: Count the # of occurrences of different q-grams in two sequences.
- Many common q-grams → small distance (independent where they occur)
- Fixed length.

q-gram Profile

q-gram Profile of u is the function $P_{u,q} : A^q \to \mathbb{N}$ s.t.
$\forall w \in A^q$, $P_{u,q}(w)$ is the number of different positions in u where w occurs.

eg: $A = \{a, c\}$ / $q = 2$ $u = acaca$: $ac \mapsto 2$, $ca \mapsto 1$, $aa \mapsto 0$, $cc \mapsto 0$

One chooses $q \ll n$ eg: $3 \le q \le 11$ ⇒ entire bacterial genome

- lexicographic ordered list:

$$[P_{u,q}(w_0), P_{u,q}(w_1), \dots, P_{u,q}(w_{r^q-1})], \quad r = |A|$$

$$\#\ q\text{-grams} = r^q - 1.$$

q-gram distance

differences in the profile list.

q-gram distance $qgdist(u,v)$:

$$qgdist(u,v) = \sum_{w \in A^q} |P_{u,q}(w) - P_{v,q}(w)|$$

→ symmetry & triangle inequality holds
→ zero not properly ind. ⇒ No Metric

GSA

q-gram

using q-grams as indices.

$A = \{a_1, \dots, a_r\}$ an ordered alphabet s.t. $a_1 < \dots < a_r$

$$\overline{a_\ell} = \ell - 1$$

is code of a_ℓ and

$$\overline{w} = \sum_{i=1}^{q} \underbrace{\overline{w[i]}}_{\text{char code}} \cdot \underbrace{r^{q-i}}_{\text{weight}} \qquad \min \quad \max \quad \in \{0, \dots, r^q - 1\}$$

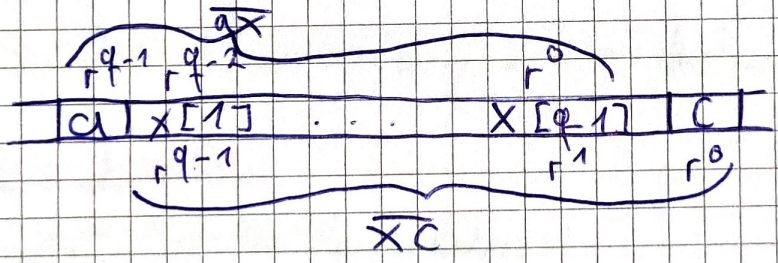
is the ord of $w \in A^q$

$|A^q| = r^q$, there are r^q numbers in $[0, r^q - 1]$

The mapping from q-grams to integer codes is bijective.

Each code of a qq q-gram in a sequence can be computed incrementally in constant time:

$$\overline{xc} = (\overline{ax} - \overline{a} \cdot r^{q-1}) \cdot r + \overline{c}$$



GUSA

Algorithm q-gram

compute the q-gram distance:

1) Accumulate the q-gram profiles of u and v in two arrays τ_u and τ_v:

$$\tau_u[\bar{w}] = P_{u,q}(w) \quad \& \quad \tau_v[\bar{w}] = P_{v,q}(w)$$

$\forall w \in \Sigma^q$

$O(m+n+\Gamma^q)$ Time, $O(\Gamma^q)$ Space

2) Compute the set $C = \{\bar{w} \mid w \text{ is } q\text{-gram of } u \text{ or } v\}$, $O(m+n)$ Space

3) Compute $qgdist(u,v) = \sum_{c \in C} |\tau_u[c] - \tau_v[c]|$

Space: $O(\underbrace{m + n}_{C} + \underbrace{\Gamma^q}_{\tau_u \& \tau_v})$

Runtime: $O(m+n+\Gamma^q)$

if $\Gamma^q \in O(m+n)$, this method is optimal

qgdist & edist

Lower bound for edist:

$$qgdist(u,v) / (2 \cdot q) \leq edist_S(u,v)$$

qgdist can be used as filter for unitedist with threshold k like unitdist.

GdA

Match Count u, v

match count = # übereinstimmende Zeichen in q-wod paaren

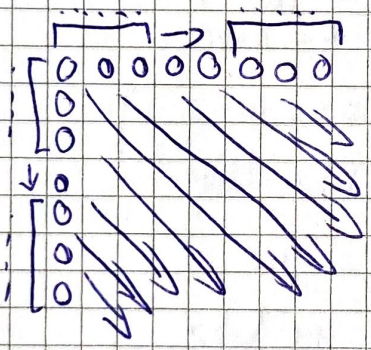
eg: u[1..3] & v[1..3] ⇒ 2
= aaa = baa

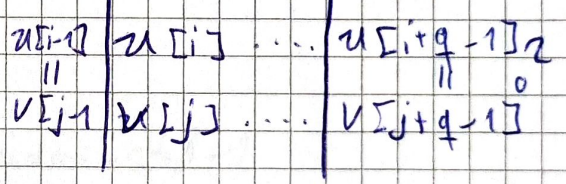
$MC(i,j) = mc(u[i+1 \dots i+q], v[j+1, \dots j+q])$
↳ $O(m\,n\,q)$ naive Lösung

besser $O(m \cdot n)$: 1) Caculate first row & first column.

$$MC(i,j) = MC(i-1, j-1) - early_match + new_match$$

early_match = 1 if $u[i-1] = v[j-1]$ else 0
new_match = 1 if $u[i+q-1] = v[j+q-1]$ else 0





GrSA

Gewichtete Substring-Matching Problem

$u \in A^*$, Gewicht $\sigma(u) = \sum_{i=1}^{|u|} \sigma(u[i])$

$\rightarrow$: $S \in A^n$, $w > 0$ vorgegebenes Gewicht

bestimme:

$$Sol(\sigma, S, w) = \{(j,l) \mid 1 \le j \le n, 1 \le l \le n+1-j, \sigma(S[j..j+l-1]) = w\}$$

$O(|S|)$:

```
w.
c_w = 0
start = 0
for i in range(len(seq)):
    c_w += σ(seq[i])
    while c_w > w:
        c_w -= σ(seq[start])
        start ++
    if c_w == w
        yield start+1, index - start+1
        c_w -= σ(seq[start])
        start ++
```

Füge dann hinzu falls $> w$ dann löschen von start bis $< w$ und wenn $== w$ dann gefunden!

GrSA

Mash: fast genome and metagenome distance estimation using MinHash

Jaccard Index

Goal: compute distance between two sequences

Common → Small Distance.

Jaccard Index: $J(U,V) = \frac{|U \cap V|}{|U \cup V|}$

- $0 \leq J(U,V) \leq 1$
- $U \cap V = \emptyset \Rightarrow J(U,V) = 0$
- $U = V \Rightarrow J(U,V) = 1$

Jaccard Index: Comparing Sequences

$G_q(U)$ und $G_q(V)$ sets of all q-grams of U & V.

↳ $J(U,V) = J(G_q(U), G_q(V))$

- common q-grams efficiently sorted:

$O(m+n)$ → for U & V enumerated encode q-grams as integers

$O(m+n)$ → store int codes in lists and sort each list with radix sort

$O(m+n)$ → find common elements in two sorted lists by merging approach (wenn werte gleich dann +1)

⇒ $O(m+n)$ space & time

Faster than q-gram distance

GST

Hash: Idea

reduces Runtime and Space

Hash:
1) computes reduced representation of q-grams of sequence (sketch)
2) estimates Jaccard Index from sketches of two sequences
3) derives distance measure from Jacard estimate

Hash Functions

store values associated with keys.

- store and search values in list $+ O(n)$
- sort keys $O(n)$ space and $O(\log n)$ time
- access $O(1)$

Hash function: maps any kind of object to a unique integer
↳ collisions can occur: C

Hash Sketch

In: Input sequence x, length q, sketch size $s \geq 1$,
hash function $h: A^q \to \mathbb{N}$

Algorithm:
- enumerate set $G_q(x)$ of all q-grams of x $O(m)$
- compute $H_{q,h}(x) = \{h(w) \mid w \in G_q(x)\}$ $O(mq)$
- keep only the s smallest values of $H_{q,h}(x)$ $O(m \log s)$
  ↳ $S_{q,h,s}(x) \to S(x)$ $\to O(m(q + \log s))$

GrSA

Hash: Hash Function

- q-gram encoding, as used for q-gram distance has no collisions
  but poly-A sequence gets smallest values in the tokens ↯

=> MinHash: (→ smallest s values)

- S(u) is a random sample of u for appropriate hash-functions
  ↳ this allows using Jaccard index
- S(u) & S(V) random samples of q-grams, the fraction of the shared items in S(u) & S(V) is expected to be similar as in u and V.

$$J_{est}(U,V) = \frac{|S(U) \cap S(V)|}{|S(U) \cup S(V)|}$$

Computation of Jest

Computation of S(u) for U = $G_q(u)$:
Time: $O(m(q + \log s))$, Space: $O(s)$

Computation of S(V) for V = $G_q(v)$:
Time: $O(n(q + \log s))$, Space: $O(s)$

Computation of $|S(U) \cap S(V)|$:
Time: $O(s)$, Space: $O(1)$

Computation of $|S(U) \cap \cup S(V)|$:
Time: $O(1)$, Space: $O(1)$

Total:
Time: $O((m+n)(q+\log s) + s)$, Space: $O(s)$

q-gram $O(m+n+rq)$: MinHash drastically improve runtime but uses less space.

GSA

All against all Jest

All-against-all: • set $T = \{t_1, \dots, t_k\}$ of k sequences
• compare all pairs t_i and t_j, $1 \le i < j \le k$

Jaccard estimate all pairs:
- precompute sketches: $O(n(q + \log s))$, $n = \sum_{i=1}^{k} |t_i|$
- Jaccard estimate $\forall$ pairs: $O(k^2 s)$

↳ Total $O(n(q + \log s) + k^2 s)$ // only one step depends on length!
↳ Fast!

Poisson Distribution

Jest → derive an approximate of the mutation rate of the sequences compared

Poisson Distribution P_λ is a discrete probability distribution with a parameter λ (the Ø # of events per interval)

$$P_\lambda(k \text{ events } \underset{\text{defined}}{\text{in interval}}) = e^{-\lambda} \frac{\lambda^k}{k!}$$

Used to quantify the probability of events which occur in a period of time or a geographical location, if:
- k # of time event occurs.
- events occur independtly
- rate at which event occur is constant
- two events never occur at the same sub-interval

Application: # mutations on a strand of DNA per unit length

GSA

Poisson for Modelling DNA-Mutations

DNA-seq $\lambda = 1$ per q-gram:

P_λ(k mutations in q-gram) : $e^{-\lambda} \frac{\lambda^k}{k!}$

Probability of matching q-grams

// Sequences differ by mutations

$\lambda = qd$, q-gram

mutations occur at single nucleotide randomly and independent with a constant rate d per base. → per q-gram $d \cdot q$

P_λ(matching q-gram) = P_λ(0 mutations in q-gram) = e^{-qd}

Probability of q-matches and Jaccard Index

- $n = \frac{|u|+|v|}{2}$ average length of u and v
- e^{-qd} single q-gram match : $\gamma = n \cdot e^{-qd}$ q-gram matches between u & v
- $J(u,v) \approx \frac{\gamma}{2n-\gamma}$ // ignore duplicates of q-grams
- $J(u,v) = \frac{e^{-qd}}{2-e^{-qd}} \Leftrightarrow \frac{2 \cdot J(u,v)}{J(u,v)+1} \approx e^{-qd}$
- $d \approx -\frac{1}{q} \ln \frac{2 \cdot J(u,v)}{J(u,v)+1}$

We can estimate the mutation rate d in terms of $J(u,v)$ & q.

GrS☆

Mutation rate and Jaccard distance

$$Jest(u,v) \approx J(u,v) \Rightarrow d \approx -\frac{1}{q} \ln \frac{2 \cdot Jest(u,v)}{Jest(u,v)+1}$$

MinHash Distance $MD_{q,s}(u,v)$ derives the mutation rate of u and v from Jest of u and v:

$$MD_{q,s}(u,v) = \begin{cases} 1, & Jest(u,v) = 0 \\ -\frac{1}{q} \ln \frac{2 \cdot Jest(u,v)}{Jest(u,v)+1}, & \text{else} \end{cases}$$

Time: $O((|u|+|v|)(q+\log s))$, Space: $O(s)$

$$MD_{q,s}(u,v) = \begin{cases} 1, & Jest(u,v) = 0 \\ -\frac{1}{q} \ln \frac{2 \cdot Jest(u,v)}{Jest(u,v)+1} \end{cases}$$

↪ mutation rate of u and v from Jest

GTSA

Gold Standard for Genome Distance

Determine: Mash Distance is a reliable distance estimator
↳ compare it to a gold standard (trusted measure)
↳ based on average nucleotide identity
$ANI(u,v)$

↳ from $SLA(u,v)$ of significant local alignments:

$$ANI(u,v) = \frac{1}{|SLA(u,v)|} \sum_{A \in SLA(u,v)} \left(1 - \frac{2\,S(A)}{|A.u| + |A.v|}\right)$$

↳ relative number of errors in alignment.

Min Hash

eg: → identification of overlaps in long reads for sequence assembly
→ all-against-all

- sketches are constant size representations of samples of q-grams from a set of sequences
  - ↳ Computation Time $\mathcal{O}(n(q + \log s))$
- ↳ used for Jaccard Index Estimation
- ↳ Jaccard Index ~ Poisson-distributed, mutation rate.
- ↳ used for Mash Distance $MD_{q,s}(u,v)$
- ↳ not model of complex evolutionary processes.
- ↳ app: large scale clustering of sequences sets

GSA

Database Search Methods

Fasta

Designed for:

w a query sequence, S be a set of sequences,

→ find all sequences in S, which have local similarities with w and to display these similarities in form of alignments and their positions within the sequence

Idea: eliminate many sequences by process only a few with SW-Algorithm.

Filter (similarity criterion):

1) One must be able to quickly determine the database sequences satisfying or not the criterion.
2) # of sequences satisfying the criterion must be very small
3) sequences not satisfying the criterion do not have high local similarities.

Finding Hot Spots (6 DNA, 2 Protein)

fixed $w \in S$, q fixed constant

Idea: Count for each diagonal the # of common q-grams in v and w.

<→ hot spots → # hot spots ≙ score for the diagonal

$m = |v|$, $n = |w|$, For d, $-m \le d \le n$:

$$hotsp(v,w,d) = |\{(i,j) \mid 1 \le i \le m-q+1,\ 1 \le j \le n-q+1,\ j-i = d,\ v[i..i+q-1] = w[j..j+q-1]\}|$$

→ # of matching q-grams in v and w whose start position pair (i,j) is on diagonal d.

$$score_{fasta}(v,w) = \max\{hotsp(v,w,d) \mid -m \le d \le n\}$$

GSA

Fasta

start positions of the matching q-grams have the same distance in both u & v.

↳ Diff: to q-gram distance model, where the order is not important

- First preprocess the query seq w

Computing score_fasta(u,v)

1) Encode ∀ q-gram as integer c, $0 \leq c \leq r^q - 1$, $r = |A|$

2) Preprocess query seq w into a table h_w, s.t. ∀c, $0 \leq c \leq r^q - 1$ } $O(n + r^q)$

$h_w(c) = \{i \mid 1 \leq i \leq |w| - q + 1, c = \overline{w[i \dots i+q-1]}\}$

↳ positions in w, where the q-gram code c occur.

↳ Can be done in $\mathcal{O}(|h_w(c)|)$ time

3) Process Database:

$n \leftarrow |w|$

for all $u \in S$ do:

$m \leftarrow |u|$

for $d \leftarrow -m$ to n do:

$hotsp(u, w, d) \leftarrow 0$ // initialize

end

for $j \leftarrow 1$ to $m - q + 1$ do:

$c \leftarrow \overline{u[j \dots j+q-1]}$ // hit code of q-gram

for all $i \in h_w(c)$ do:

$hotsp(u, w, j-i) \leftarrow hotsp(u, w, j-i) + 1$

end

end

$score_{fasta}(u, w) = \max\{hotsp(u, w, d) \mid -m \leq d \leq n\}$

end

GASA

Fasta

Preprocessing w:

1) First Scan Compute $|h_w(c)|$ & $P(c)$

$$P(c) = \sum_{c' \leq c} |h_w(c')|$$

2) Compute H, with $|H| = P(r^q - 1) + |h_w(r^q - 1)|$
   → all positions!

Second Scan

$P(c)$: aa ac ca cc ... 13
0 3 8 13 $H(c)$

[a a] a c ca $H(P(aa)) = 1$ & $P(aa)++$ 0|1

a [a a] c ca $H(P(aa)) = 2$ & $P(aa)++$ 1|2

a a [a c] ca $H(P(ac)) = 3$ & $P(ac)++$ 3|3

[o o o]

↳ eg: $H[8, \ldots, 12]$ contains $\{5, 7, 9, 11, 14\}$ for ca occurences!

$O(r^q + m + n + \sum_{d=-m}^{n} \text{hostsp}(x, w, d))$ for one alphabet seq x.

The most common q-grams the layer computations!
↳ no time wasting on filtered out sequences!

GASA

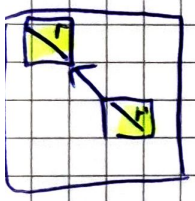
Caching hot spots for diagonal runs

score for $(u, w) \leq k$ threshold ⇒ discard

Else:

- looking for diagonal runs in the Edit-distance matrix.
  - ↳ hotspots occurring on same diagonal with small gaps between them.
- scoring diagonal runs: (+) hotspots, (-) Gaps.
  - ↳ A diagonal can contain more than one diagonal run!

Constructing a DAG from diagonal runs



Nodes of the DAG are diagonal runs with corresponding weights.

Diagonal run upper left corner $(l_1(r), l_2(r))$ and lower right corner $(h_1(r), h_2(r))$. The nodes r & r' are connected if $h_1(r) < l_1(r')$ and $h_2(r) < l_2(r')$

edges negative weight.

Compute paths of maximal weight.

↳ paths pick first & last node, upper left corner of the first node and the lower right corner of the last node define a pair of substrings of u and v. ⇒ aligned via global alignment.

↳ If score of optimal global alignment $\leq k$ ⇒ report it

GSA

Blast

Basic Local Alignment Search Tool

- 5 steps
  - reduce the search space, but each is more fine grain and take longer.

1) find blast hit
2) identify pairs of hits on the same diagonal
3) perform ungapped extension
4) perform gapped extension
5) collect traceback information and display alignments

Stage 1: find Blast Hit

each database seq is compared with query seq.

eg: and allow indels

Blast Hit

$q \in \mathbb{N}, k \geq 0$, A blast-hit is a pair (i, j) of indices s.t.:

$$score_\sigma(x[i..i+q-1], y[j..j+q-1]) \geq k.$$

GSA

Calculate Blast Hit

Find blast hits over all database seq's.

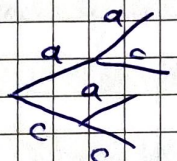
(1) preprocess query sequence w. non-exact matches.

Construct k-environment:

$$Env_k(w) = \{(s,j) \mid s \in \Lambda^q, 1 \le j \le |w| - q + 1, score_\sigma(s, w[j \dots j+q-1]) \ge k\}$$

↳ ∀ positions in w the q-grams s achieving a score ≥ k, when comparing to the q-gram at that position.

↳ Construct a trie?



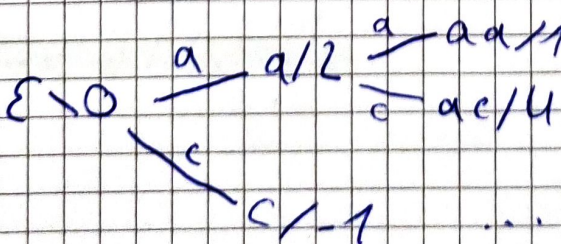
And only look at subtries, that already have a big enough score.

↳ trie node α represents sequence y of length q', q' ≤ q

↳ maintain: $score_\sigma(y, x[1 \dots q'])$ x is used q-gram of w.

↳ Each node in the trie at depth q with $score_\sigma \ge k$ represents a sequence of $Env_k(w)$.

↳ Stop looking at subtrie if score ≥ k can not be reached anymore!



GCS☆

Calculate 3last Hit

(2) represent each $s \in \Delta^q$ as unique integer C

$0 \leq C \leq r^q - 1$, $r = |\Delta|$

(3) $Env_k(w)$ represented as follows:

- a vector V of r^q bits st: $V[\bar{s}] = 1$ if $(s, j) \in Env_k(w)$ for some j
- $\forall s \in \Delta^q$ with $V[\bar{s}] = 1$, store $Pos(s) = \{j \mid (s, j) \in Env_k(w)\}$
  ↳ like in Fast Rank.

(4) $\forall$ database sequences x

↳ processed by shifting a window of length q over it.

↳ let s be the current substring $s = x[i \ldots i+q-1]$

↳ compute $\bar{s}$

↳ look up if $V[\bar{s}] \stackrel{?}{=} 1$

↳ Yes, there is some j, such that $(s, j) \in Env_k(w)$

↳ found in $j \in Pos(s)$.

↳ $\forall j \in Pos(s) \rightarrow (i, j)$ is a 3last hit

(1) - (3) depends on w (4) only on x

only once!

Once w is preprocessed, x is processed in $\mathcal{O}(|x| + z)$ time

z = # 3last hits

GASA

Stage 2: Identify pair BLAST hits on the same diagonal

Look for pair of blast hits on the same diagonal with same maximal distance δ.

→ 2 hits (i,j) & (i',j') • same diagonal with $d = j-i = j'-i$

↳ if $i < i'$: $i + k = i'$, $k > 0$

$j + k = j'$.

→ With respect to the same diagonal, blast hits are computed in increasing order of the positions in both sequences and with the distance k.

↳ $f(d) = i$ s.t. blast hit at $d = j-i$.

↳ for new blast hit (i',j') check: $i' - f(d) \leq \delta$

$\delta \triangleq$ maximal distance on diagonal

↳ $(f(d), d+f(d))$ & (i',j') on d is at most δ

↳ relevant blast hit pair

↳ next $f(d) = i'$

GSA

Stage 3: Ungapped Extensions

5 last hit pair with max distance δ.

→ Determine if the pair occurs in a region of pairwise similarity:
↳ Mid point between hits, starting point for ungapped extensions.

↳ Sensitivity depends on a "drop-off" $X_d > 0$

→ Mid point (i, j) between two blast hits:

- $m = |u|$, $n = |w|$
- extension to the left: $u[1..i-1]$ & $w[1...j-1]$ are compared right-to-left
- extension to the right: $u[i...m]$ & $w[j...n]$ are compared left-to-right
- Each char pair $(u[i-\ell], w[j-\ell])$ & $(u[i+r], w[j+r])$
  $1 \le \ell \le \min\{i-1, j-1\}$, $0 \le r \le \min\{m-i, n-j\}$
  ↳ score

→ calculate scores and keep track of X_{max} reach during extension
Stopp: if $X_{max} - X_d$ is reached

⇒ The pair of sequences delivered by the extensions (l, r) ~~are~~ is called maximum segment pair MSP.

↳ ∀ MSPs a significant score is computed
↳ if this is better than predefined significance threshold
↳ next stage

GASA

Stage 4: Gapped Extension

Determine if the high scoring ungapped region forms part of a longer, higher scoring alignment.

→ Explore only I & D that augment the high-scoring ungapped alignment.

↳ Identify seed point: lies within a high scoring portion of the ungapped region.

↳ Use affine gap scores.

→ Stops when score falls below drop of γ_d

→ When overall score is greater than some value → next stage

Stage 5: Collect traceback information and display alignments

Karlin-Altschul statistics to report the statistical significance of each gapped local alignment as an E-Value.

E.-Value = # of local alignments with at least the same score that one expects when comparing a random query sequence with typical amino-acid composition against a random database sequence.

Steps:

1) Nominal Score S, $\forall$ local alignments based on a scoring matrix and gap cost

2) Normalized Score $$S' = \frac{\lambda S - \ln K}{\ln 2}$$

λ & K are pre computed by random simulation!

3) E-Value:
$$E = \frac{Q}{2^{S'}} = \frac{mn}{2^{S'}}$$

↳ used to determine the cutoff parameter from the E-Value